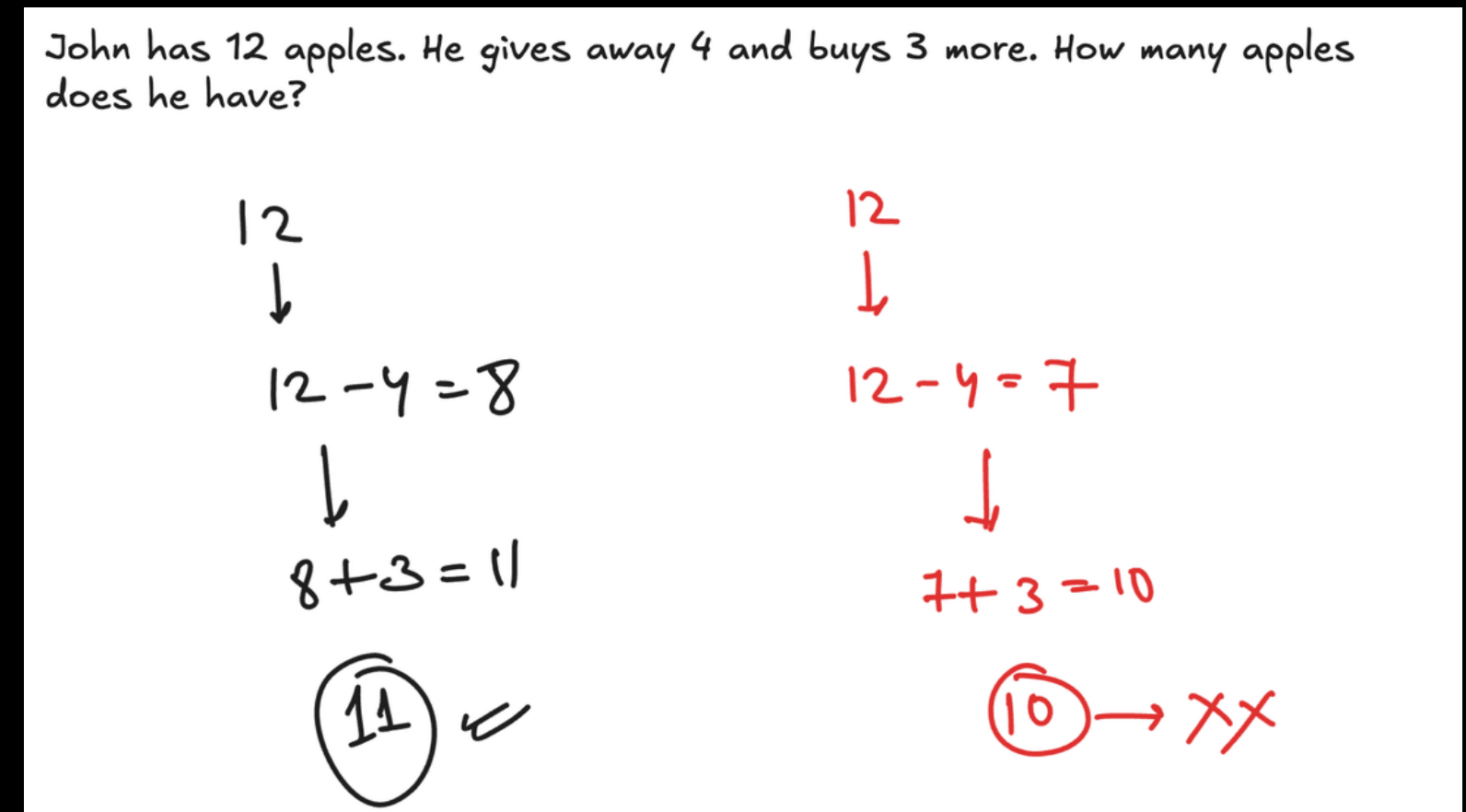
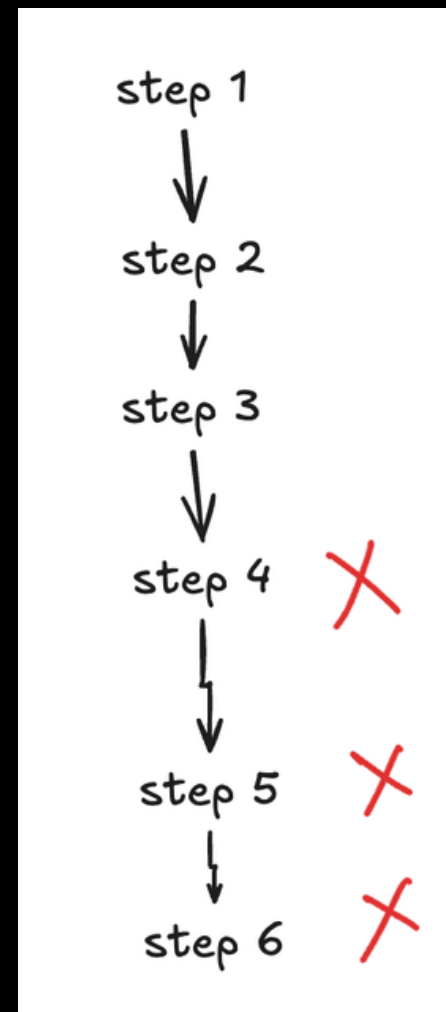


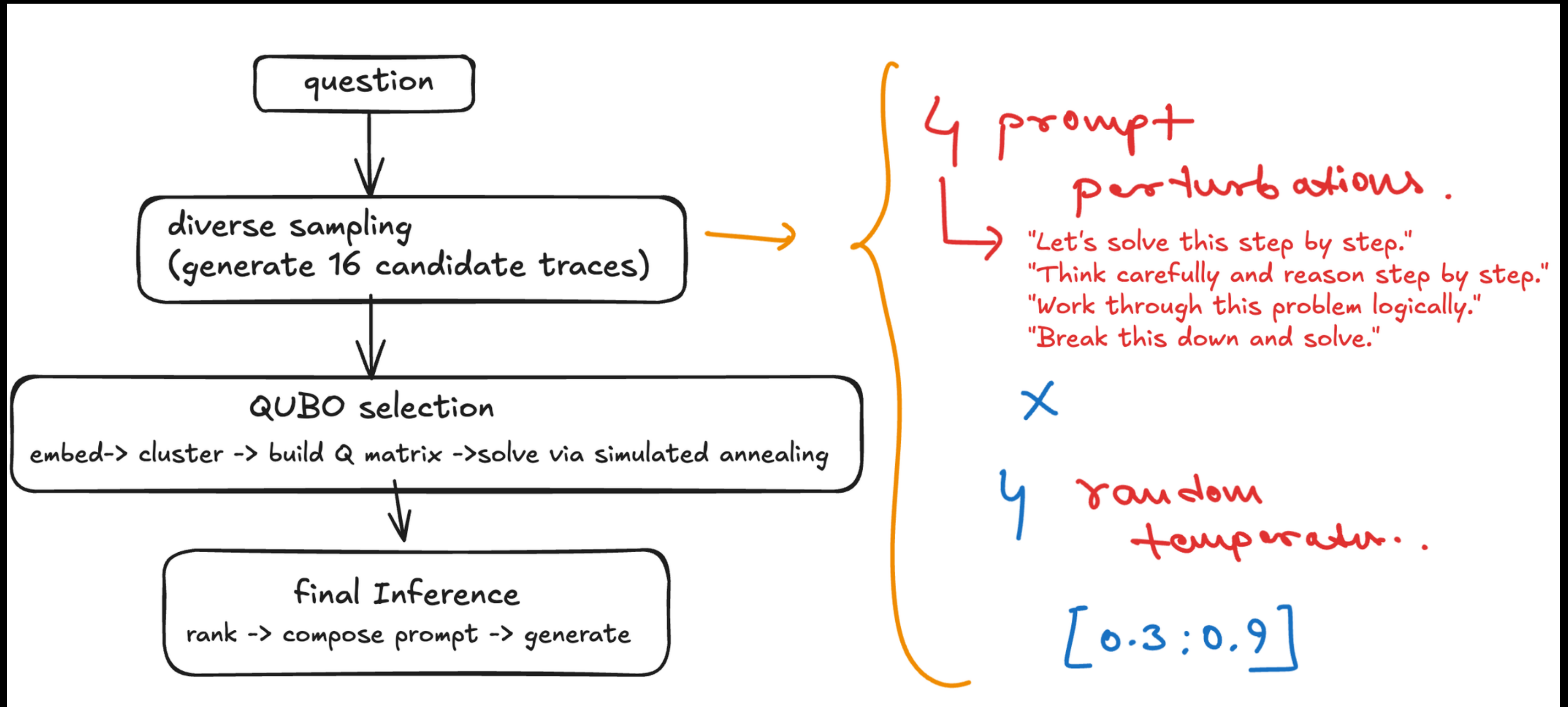
Samsung Prism_3

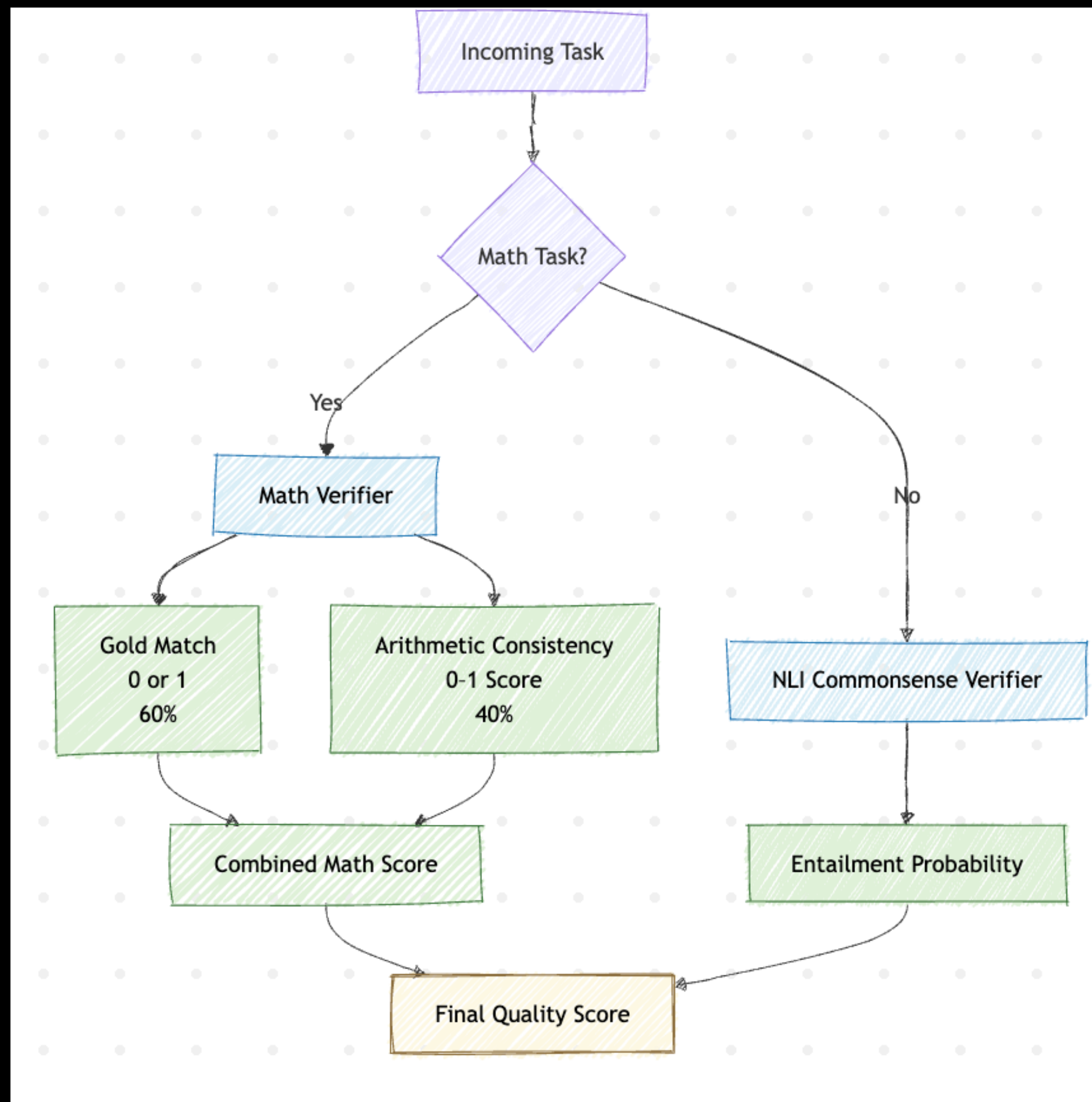
Why do we need QUBO?

- **Existing SLMs (1-3B) have three fundamental problems that QUBO addresses:**
- **1. Single-path brittleness** – One greedy/CoT pass makes a mistake and can't recover; error propagates through the reasoning chain
- **2. Self-consistency/majority vote keeps redundant wrong answers** – It aggregates many traces, but popular-but-wrong answers dominate and semantically similar traces overtake, giving no diversity signal
- **3. No explicit quality-diversity tradeoff** – Top-k picks individually good traces but ignores that they may all say the same thing (redundant)



How Qubo Helps??





Maths Scoring Logic

Final Score = $(0.6 \times \text{Gold Match}) + (0.4 \times \text{Arithmetic Consistency})$



```
def _extract_last_number(self, text: str) -> Optional[float]:
    cleaned = text.strip().replace(",", "")
    matches = re.findall(r"-?\d+(?:\.\d+)?", cleaned)
    if not matches:
        return None
    try:
        v = float(matches[-1])
        return int(v) if v.is_integer() else v
    except ValueError:
        return None
```

`gold_match = 1.0` $\rightarrow$ If $|\text{pred} - \text{gold}| < 0.01$

`gold_match = 0.0` $\rightarrow$ If $|\text{pred} - \text{gold}| \geq 0.01$ (or if no numbers could be found at all)

Student A 12 $\rightarrow 12$

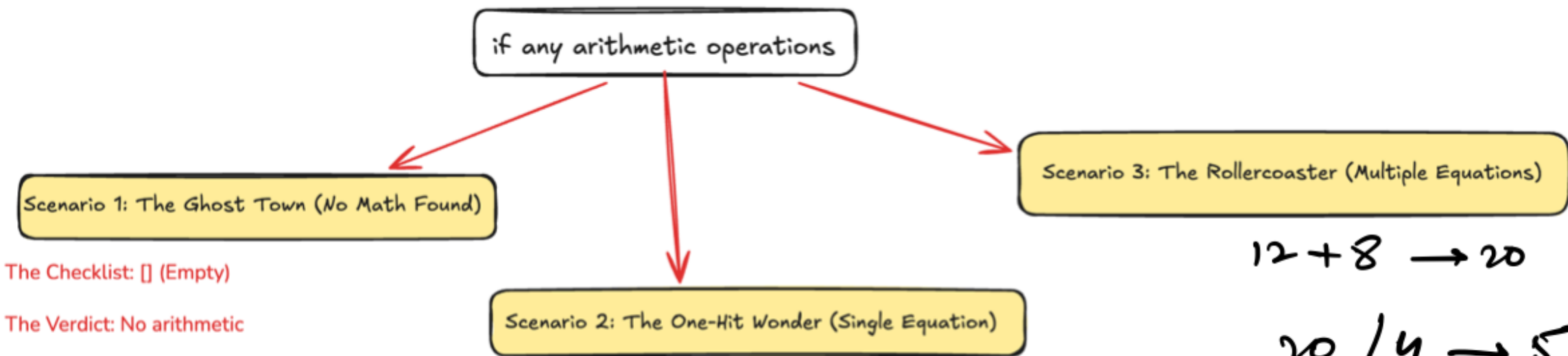
Benchmark
(Gold answer) $\rightarrow 12$

$$|12 - 12| = 0 < 0.01$$

$\downarrow$
TRUE

gives a
perfect
score of
1.0

Arithmetic Consistency



The Checklist: [] (Empty)

The Verdict: No arithmetic

The Score: default score: base = 0.3

The Checklist: [4.0]

The Verdict: One equation cannot contradict itself. The student is perfectly consistent.

The Score: base = 1.0

$$12 + 8 \rightarrow 20$$

$$20 / 4 \rightarrow 5$$

Consistent [20.0, 5.0]

Penalty formula

$$\text{Consistency} = \frac{1.0}{1.0 + \text{mean}(\text{diffs})}$$

$$\frac{1}{1 + \text{mean}(20-5)} = \frac{1}{1+15} = \frac{1}{16} = 0.0625$$

$0.6 \times \text{Gold-maths} + 0.4 \times \text{math consistency.}$

$\left| \text{predict. answer} - \text{gold answer} \right| \begin{cases} < 0.01 \rightarrow 1.0 \\ \geq 0.01 \rightarrow 0.0 \end{cases}$

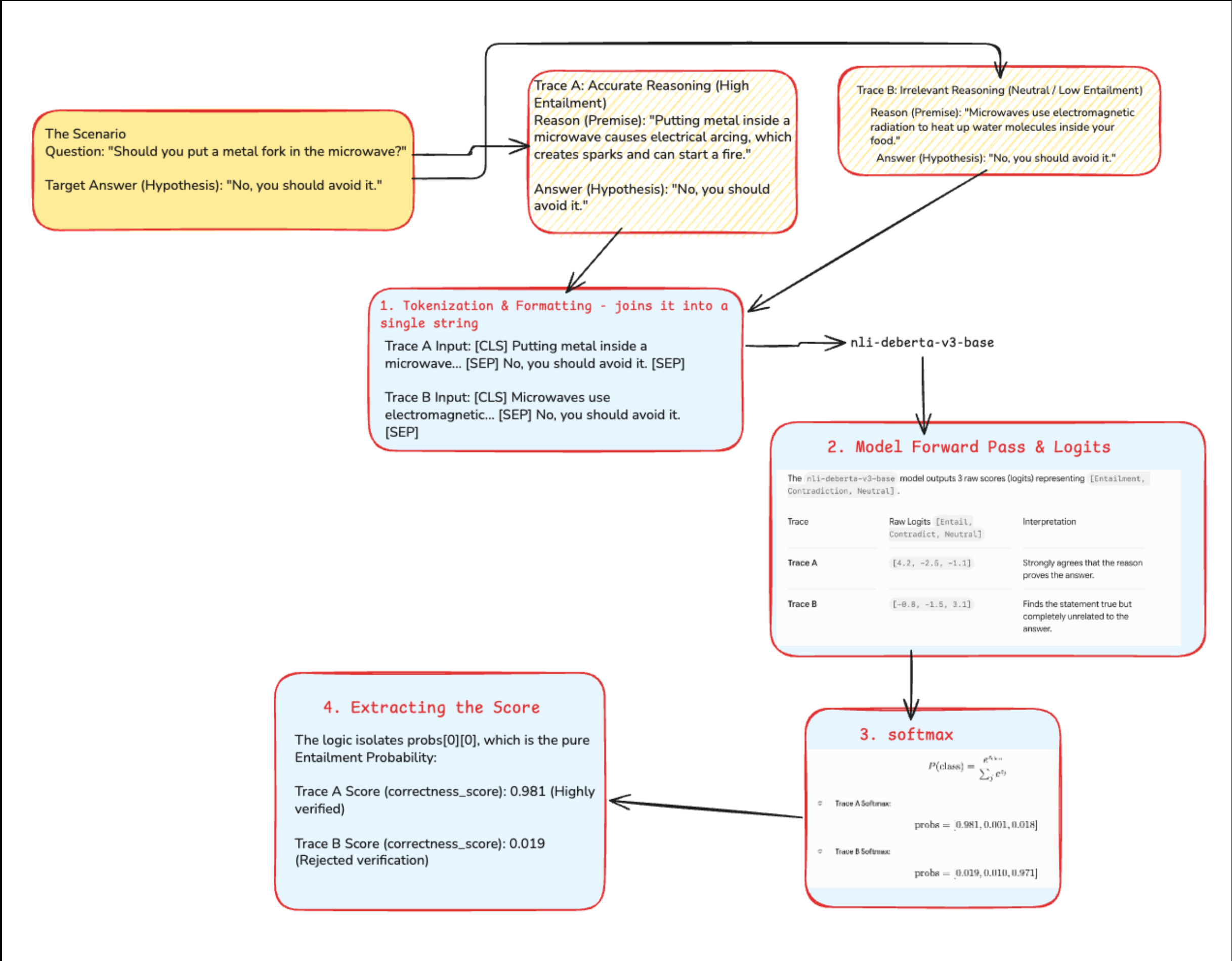
$\text{base} = 0.3$ (if no arithmetic / maths equation.)

Single calculation (nothing to contradict $\rightarrow 1.0$)

many / multiple steps.

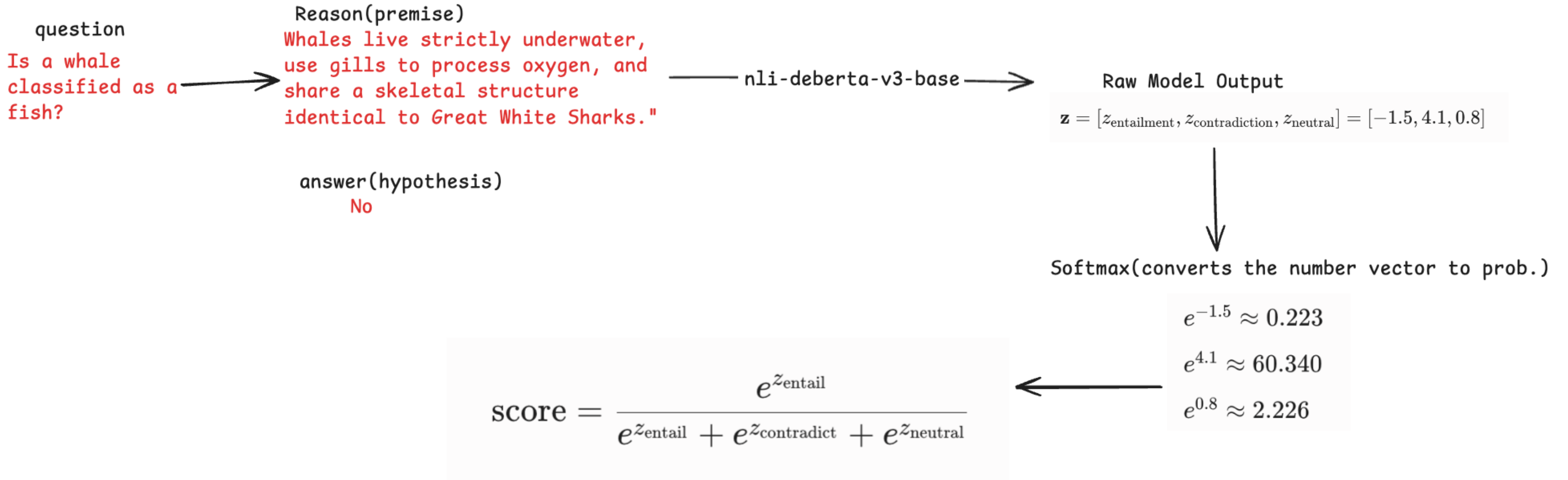
$0.4 \times \left(\frac{1.0}{1.0 + \text{mean}(\text{diffs})} \right)$

Non-Arithmetic Questions



How it works:

1. Take the candidate's reason (e.g., "Penguins cannot fly because their wings are adapted for swimming")
2. Take the candidate's answer (e.g., "No")
3. Feed (premise=reason, hypothesis=answer) into cross-encoder/nli-deberta-v3-base
4. Model outputs 3 raw logits: [entailment, contradiction, neutral]
5. Apply `torch.softmax()` → probabilities summing to 1.0
6. Return `probs[0][0]` – the probability of entailment



Production Ready??

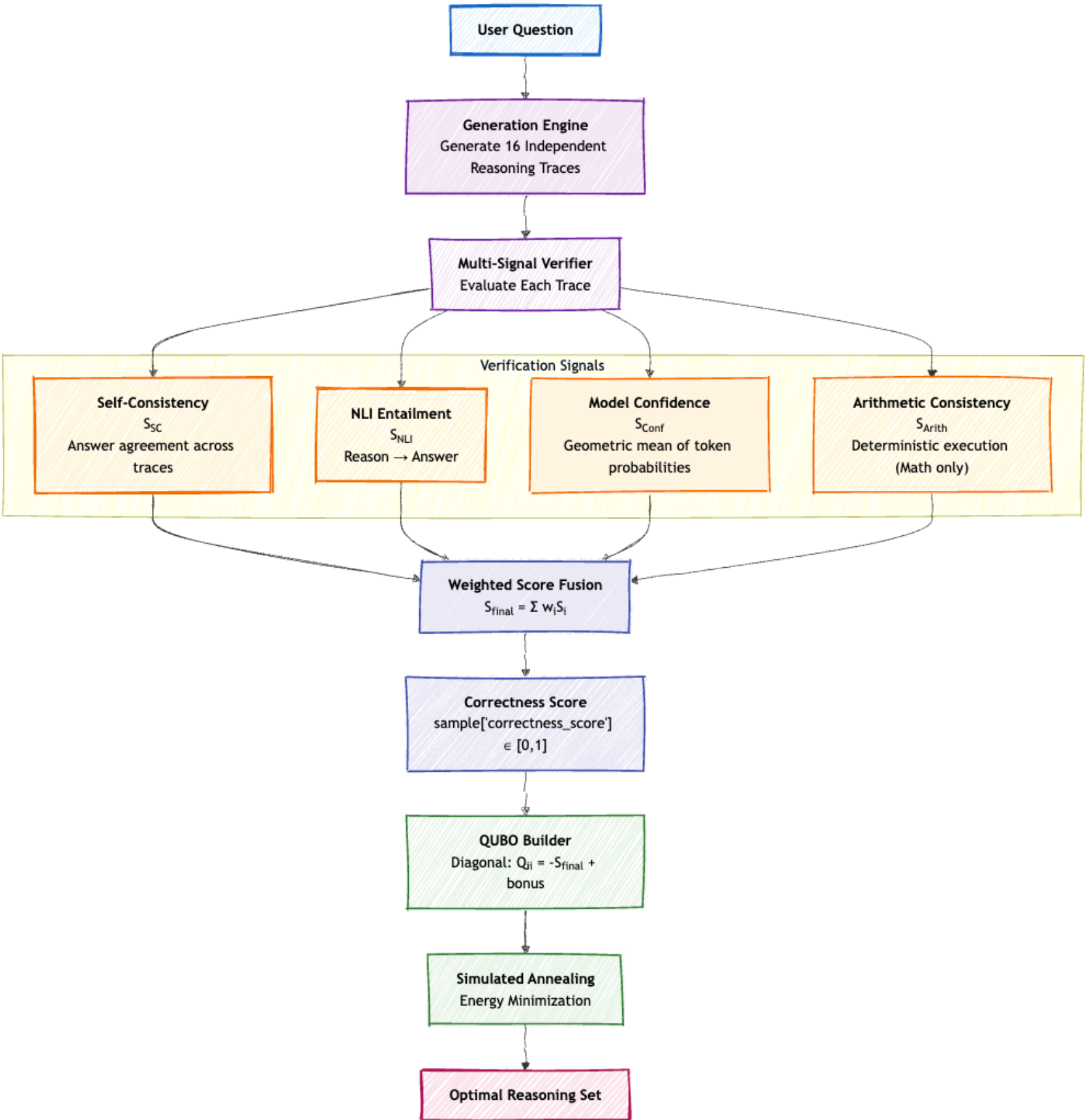
No Ground Truth required	Ground Truth Required
Yes/No questions – e.g., "Can penguins fly?", "Is Paris the capital of France?"	Math word problems
Commonsense reasoning – e.g., StrategyQA, "Will a rock sink in water?"	Any task where the answer is a specific string/number
MCQ choice selection – e.g., MMLU, ARC-Challenge (reason entails letter A/B/C/D)	
BBH subtasks – boolean expressions, causal judgement, date understanding, etc.	

For non-ground-truth questions, the score is purely the NLI entailment probability — no gold match, no weighted combination.

$$\text{score} = \frac{e^{z_{\text{entail}}}}{e^{z_{\text{entail}}} + e^{z_{\text{contradict}}} + e^{z_{\text{neutral}}}}$$

Solution

$$S_{\text{final}} = w_{\text{SC}}S_{\text{SC}} + w_{\text{NLI}}S_{\text{NLI}} + w_{\text{Conf}}S_{\text{Conf}} + w_{\text{Arith}}S_{\text{Arith}}$$



1. Self consistency

Final Answer Extracted	Sample Count	Example Generation Snippet
14 (Correct)	10	"...Left with 3. Buys $3 \times 3 = 9$. Total is $3 + 9 = 14$."
11 (Math Error)	4	"...Left with 3. Buys $3 \times 3 = 9$. Total is $2 + 9 = 11$."
9 (Logic Error)	2	"...Left with 3. Buys 3 times left, which is 9. Total is 9."

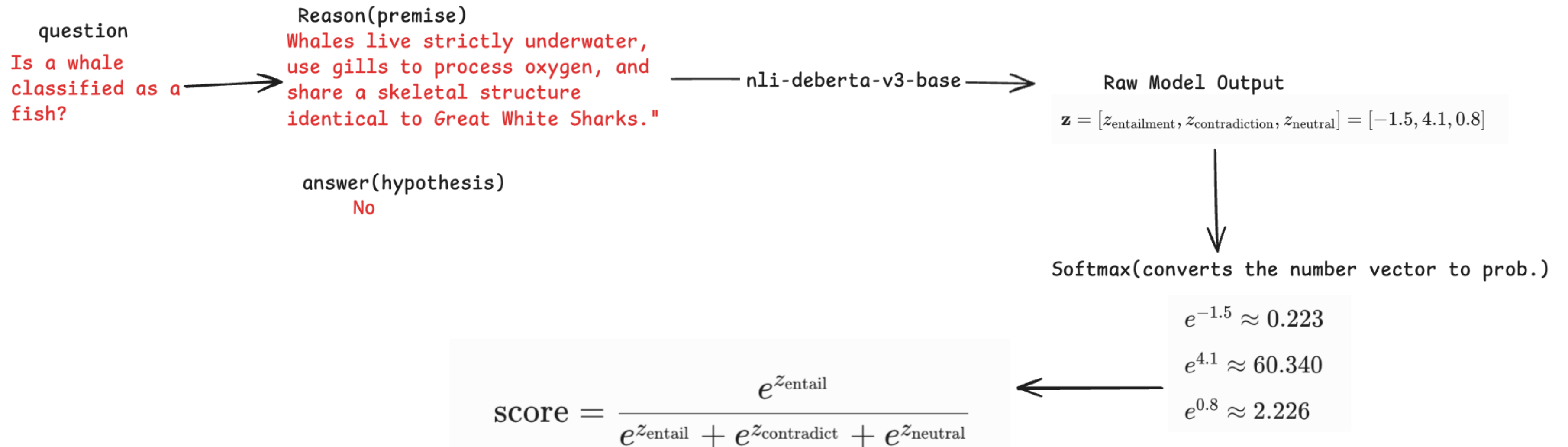
$$S_{\text{SC}} = \frac{\text{Frequency of Target Predicted Answer}}{\text{Total Number of Sampled Traces}}$$

$$S_{\text{SC}} = \frac{10}{16} = \mathbf{0.625}$$

2. NLI

How it works:

1. Take the candidate's reason (e.g., "Penguins cannot fly because their wings are adapted for swimming")
2. Take the candidate's answer (e.g., "No")
3. Feed (premise=reason, hypothesis=answer) into cross-encoder/nli-deberta-v3-base
4. Model outputs 3 raw logits: [entailment, contradiction, neutral]
5. Apply `torch.softmax()` → probabilities summing to 1.0
6. Return `probs[0][0]` – the probability of entailment

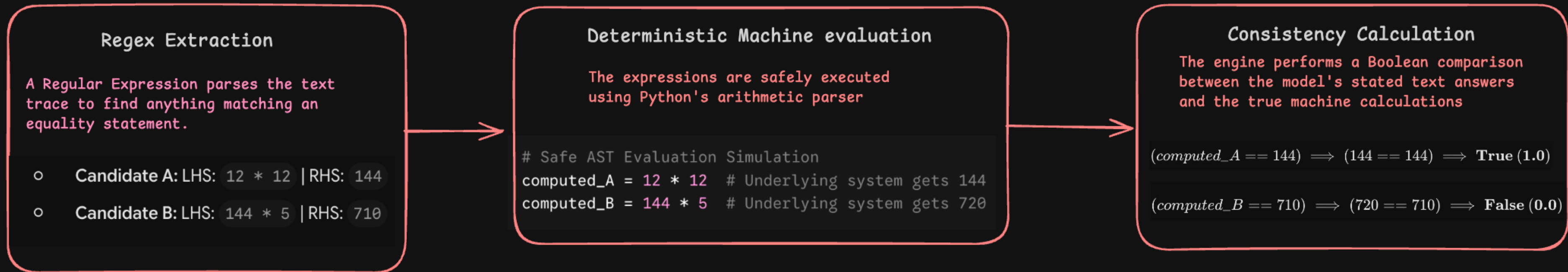


3. Length-Normalized Model Confidence

When an LLM generates a reasoning trace, it doesn't just output words it outputs a probability distribution for every single token it selects. Model confidence taps into these raw probabilities to see how "sure" the model felt during generation.

$$S_{\text{Conf}} = \exp \left(\frac{1}{N} \sum_{i=1}^N \log P(t_i) \right)$$

4. Arithmetic Consistency



$$S_{\text{Arith}} = \frac{\sum \text{Passed Assertions}}{\text{Total Assertions Identified}} = \frac{1 + 0}{2} = \mathbf{0.500}$$

$$S_{\text{final}} = w_{\text{SC}}S_{\text{SC}} + w_{\text{NLI}}S_{\text{NLI}} + w_{\text{Conf}}S_{\text{Conf}} + w_{\text{Arith}}S_{\text{Arith}}$$

How the Weights are decided??

The Two Strict Rules

To ensure the final score stays perfectly between 0.0 and 1.0 , the system enforces:

- 1. **Sum to 1:** All active weights must add up exactly to 1.0 ($\sum w_i = 1$).
- 2. **No Negatives:** Every individual weight must be greater than or equal to 0.

2. How the Weights Are Decided

While the production system runs **unsupervised** (without answer keys), the weights themselves are calculated **offline during development** using a small validation dataset *with* answers.

Engineering teams typically use one of two quick approaches:

- **The Data-Driven Approach (Optimization):** You feed a few hundred validation questions through the pipeline. You then use an optimization library (like Optuna) or simple Logistic Regression to find the exact weights that give the highest-scoring traces the correct answers. These optimized weights are then hardcoded into your production config.
- **The Heuristic Approach (Task Routing):**
If empirical optimization data isn't available yet, weights are assigned based on the task type using a fallback matrix:

Task Type	wSC (Consensus)	wNLI (Logic)	wConf (Confidence)	wArith (Math)
Math / Quantitative	0.4	0.3	0.2	0.1
Commonsense / Verbal	0.5	0.3	0.2	0.00 <i>(Disabled)</i>

What is QUBO?

Quadratic Unconstrained Binary Optimization is a mathematical framework used to find optimal choices from a discrete set of possibilities.

qubo equation

$$\min_{\mathbf{x} \in \{0,1\}^n} E(\mathbf{x}) = \mathbf{x}^T Q \mathbf{x}$$

=

$$E(\mathbf{x}) = - \sum_{i=1}^n S_i x_i + \lambda \sum_{i < j} C_{ij} x_i x_j$$

S_i : The individual correctness score of trace i (higher is better).

C_{ij} : The semantic cosine similarity between trace i and trace j .

λ : The penalty scaling weight (hyperparameter).

x_i, x_j : Binary decision variables (1 = keep, 0 = discard).

$i < j$: Summation constraint checking each unique pair exactly once (upper-triangular formulation).

scores generated by the logic before

Individual Correctness Scores (S_i)

Trace 1 (x_1): "Paris is the capital of France because it is the seat of the government."

$S_1 = 0.90$ (Highly accurate logic)

Trace 2 (x_2): "Paris is France's capital city, which hosts the famous Eiffel Tower."

$S_2 = 0.85$ (Highly accurate, but redundant with Trace 1)

Trace 3 (x_3): "Lyon is a historic culinary city located in southern France."

$S_3 = 0.05$ (Irrelevant to the direct question)

Scenario A: The Naive Greedy Choice — $\mathbf{x} = [1, 1, 0]$

The system tries to select both high-scoring traces (Trace 1 and Trace 2), ignoring their redundancy.

1. Linear Term ($-\sum S_i x_i$):

$$-(S_1 \cdot 1 + S_2 \cdot 1 + S_3 \cdot 0) = -(0.90 + 0.85) = -1.75$$

2. Quadratic Term ($+\lambda \sum_{i < j} C_{ij} x_i x_j$):

$$\lambda \cdot (C_{12} x_1 x_2 + C_{13} x_1 x_3 + C_{23} x_2 x_3)$$
$$1.50 \cdot (0.80 \cdot 1 \cdot 1 + 0.10 \cdot 1 \cdot 0 + 0.15 \cdot 1 \cdot 0) = 1.50 \cdot 0.80 = +1.20$$

3. Total Energy:

$$E([1, 1, 0]) = -1.75 + 1.20 = -0.55$$

Scenario B: Including an Irrelevant Trace — $\mathbf{x} = [1, 0, 1]$

The system tries to select the best trace (Trace 1) along with the irrelevant trace (Trace 3).

1. Linear Term ($-\sum S_i x_i$):

$$-(S_1 \cdot 1 + S_2 \cdot 0 + S_3 \cdot 1) = -(0.90 + 0.05) = -0.95$$

2. Quadratic Term ($+\lambda \sum_{i < j} C_{ij} x_i x_j$):

$$1.50 \cdot (C_{12} \cdot 1 \cdot 0 + C_{13} \cdot 1 \cdot 1 + C_{23} \cdot 0 \cdot 1) = 1.50 \cdot 0.10 = +0.15$$

3. Total Energy:

$$E([1, 0, 1]) = -0.95 + 0.15 = -0.80$$

Scenario C: The Ideal Optimized Choice — $\mathbf{x} = [1, 0, 0]$

The system selects only the single best representative path (Trace 1) and cuts out everything else.

1. Linear Term ($-\sum S_i x_i$):

$$-(S_1 \cdot 1 + S_2 \cdot 0 + S_3 \cdot 0) = -0.90$$

2. Quadratic Term ($+\lambda \sum_{i < j} C_{ij} x_i x_j$):

Since $x_2 = 0$ and $x_3 = 0$, every interaction term evaluates to 0.
$$1.50 \cdot (0) = 0.00$$

3. Total Energy:

$$E([1, 0, 0]) = -0.90 + 0.00 = -0.90$$

The solver evaluates different combinations of $\mathbf{x} = [x_1, x_2, x_3]$ to see which one results in the lowest total energy $E(\mathbf{x})$.

scores calculated
in previous steps

$$E(\mathbf{x}) = -\sum_{i=1}^n S_i x_i + \lambda \sum_{i < j} C_{ij} x_i x_j$$

Penalty Hyperparameter

- $\lambda = 1.50$

cosine similarities

Pairwise Semantic Similarities (C_{ij})

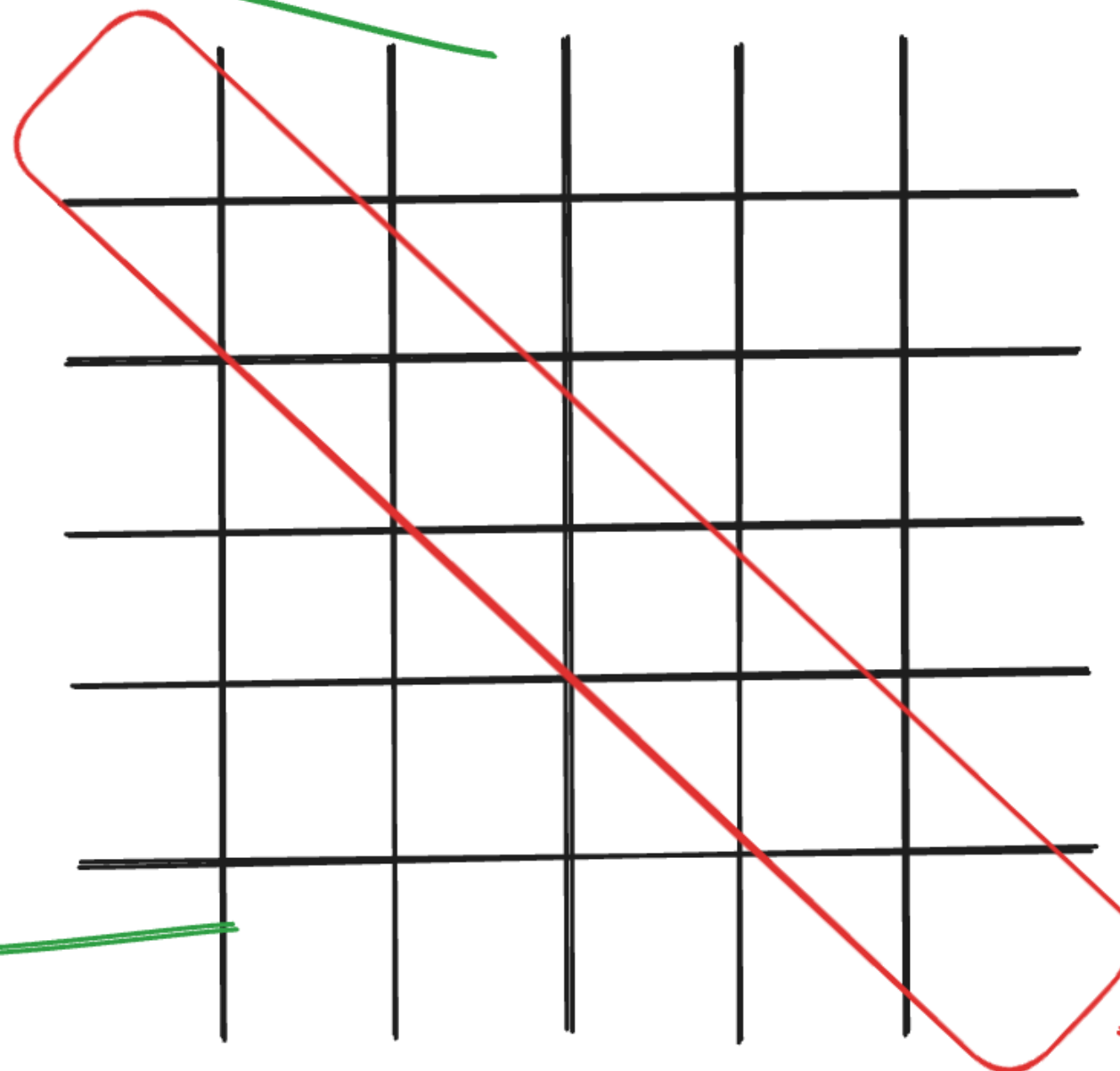
- C_{12} (Trace 1 & 2): 0.80 (Extremely high semantic redundancy)
- C_{13} (Trace 1 & 3): 0.10 (Very low semantic overlap)
- C_{23} (Trace 2 & 3): 0.15 (Very low semantic overlap)

SCENARIO TESTED	BINARY STATE VECTOR (X)	RESULTING ENERGY (E)	VERDICT
Greedy Redundancy	[1, 1, 0]	-0.55	High penalty suppresses this state.
Irrelevant Inclusion	[1, 0, 1]	-0.8	Poor quality score limits optimization deepness.
Optimal Selection	[1, 0, 0]	-0.9	Global Minimum (Winner)

matrix construction from the equation

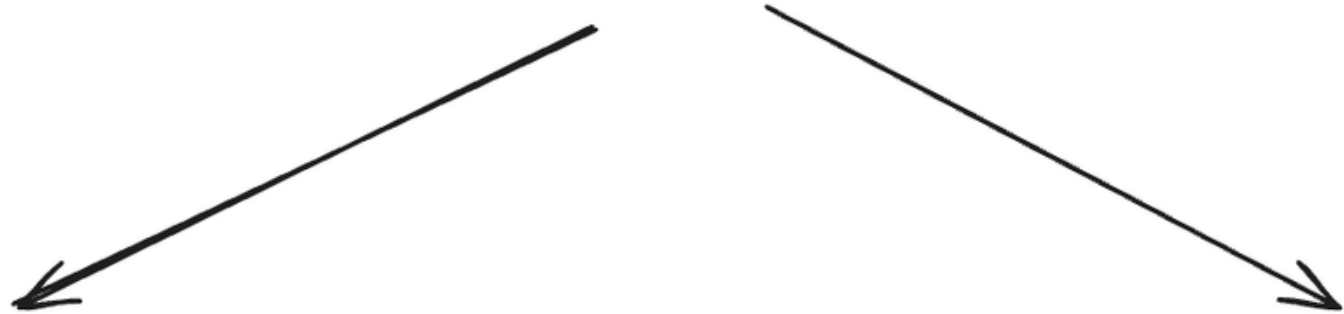
$$+ \lambda \sum_{i < j} C_{ij} x_i x_j$$

off diagonal elements



$$- \sum_{i=1}^n S_i x_i$$

2 ways to construct the matrix



Form 1: The Upper Triangular Matrix (Direct Mapping)

Here, the full penalty is loaded strictly into the top-right half of the matrix, and the bottom-left is left as zeros.

- **Diagonal (Q_{ii}):** Set to $-S_i$
- **Off-Diagonal ($i < j$):** Set to λC_{ij}
- **Lower Triangle ($i > j$):** Set to 0

$$\mathbf{Q}_{\text{triangular}} = \begin{pmatrix} -S_1 & \lambda C_{12} & \lambda C_{13} \\ 0 & -S_2 & \lambda C_{23} \\ 0 & 0 & -S_3 \end{pmatrix} = \begin{pmatrix} -\mathbf{0.90} & +\mathbf{1.20} & +\mathbf{0.15} \\ 0 & -\mathbf{0.85} & +\mathbf{0.225} \\ 0 & 0 & -\mathbf{0.05} \end{pmatrix}$$

Form 2: The Symmetric Matrix (Production Standard)

In production, we usually split the off-diagonal penalties evenly between the upper and lower halves (Q_{ij} and Q_{ji}) by dividing them by 2. This keeps the matrix perfectly balanced.

- **Diagonal (Q_{ii}):** Set to $-S_i$
- **Off-Diagonal ($i \neq j$):** Set to $\frac{\lambda C_{ij}}{2}$

$$\mathbf{Q}_{\text{symmetric}} = \begin{pmatrix} -S_1 & \frac{\lambda C_{12}}{2} & \frac{\lambda C_{13}}{2} \\ \frac{\lambda C_{21}}{2} & -S_2 & \frac{\lambda C_{23}}{2} \\ \frac{\lambda C_{31}}{2} & \frac{\lambda C_{32}}{2} & -S_3 \end{pmatrix} = \begin{pmatrix} -\mathbf{0.90} & +\mathbf{0.60} & +\mathbf{0.075} \\ +\mathbf{0.60} & -\mathbf{0.85} & +\mathbf{0.1125} \\ +\mathbf{0.075} & +\mathbf{0.1125} & -\mathbf{0.05} \end{pmatrix}$$

[1,1,0]

- Using $\mathbf{Q}_{\text{triangular}}$: $-0.90(1) - 0.85(1) + 1.20(1 \cdot 1) = -\mathbf{0.55}$
- Using $\mathbf{Q}_{\text{symmetric}}$: $-0.90(1) - 0.85(1) + 0.60(1 \cdot 1) + 0.60(1 \cdot 1) = -\mathbf{0.55}$

Final Answer Generation

optimal vector selected

- QUBO Output Vector ($\mathbf{x}$): [1, 0, 1, 0, 0, 1, 0, 1]
- Active Mapping Index Matrix: [2, 5, 8, 12, 3, 7, 14, 1]
- Retained Production Traces: samples[2], samples[8], samples[7], samples[1]



Contextual Semantic Re-Ranking

1. Generate geometric embeddings for all selected reasons using `SentenceTransformer` ($\mathbf{e}_{\text{reason}} \in \mathbb{R}^{384}$).
2. Generate the embedding for the original user query ($\mathbf{e}_{\text{question}} \in \mathbb{R}^{384}$).
3. Compute the **Cosine Similarity** between the question and each active reason.
4. Sort the reasoning steps in descending order so the most relevant, on-topic insights occupy the prime slots at the top of the prompt.



Structural Prompt Construction

default $K = 6$, prompt wrap

```
Here are some reasoning steps:  
1. {Most contextually relevant reason text}  
2. {Second most contextually relevant reason text}  
...  
K. {Least relevant selected reason text}
```

```
Based on these steps, answer the following question.  
Question: {Original User Query}  
Answer:
```

SLM

